

VINSUP SKILL ACADEMY

Coimbatore, Tamil Nadu

Project Documentation

RETAIL SALES ANALYSIS

Using Excel – SQL – Python – Power BI

SUBMITTED BY

SANDHIYA S

TABLE OF CONTENTS

S. No	Content
1.	INTRODUCTION
2.	OBJECTIVE
3.	TECHNOLOGY USED
4.	TASKS
5	KEY INSIGHTS
6.	CONCLUSION

1. INTRODUCTION

Retail Sales Analysis is the process of examining sales data to identify trends, customer behavior, product performance, and business growth opportunities. This project focuses on analyzing retail sales data using data analytics tools and visualization techniques to generate meaningful business insights. The analysis helps businesses understand sales performance, improve decision-making, and optimize overall operations. The project was developed using Microsoft Excel, SQL, Python, and Microsoft Power BI for data cleaning, analysis, visualization, and dashboard creation. The analysis transforms raw sales data into meaningful insights that help businesses optimize sales strategies, improve inventory management, and support data-driven decision-making.

2. OBJECTIVE

- To analyze retail sales data from brands, categories, customers, orders, products, stores, stocks, and staff to understand overall business performance.
- To identify sales trends, top-selling products, and profitable customer segments using data analytics techniques and visualization tools.
- To perform data cleaning, preprocessing, and analysis using Microsoft Excel, SQL, and Python for generating accurate insights.
- To create an interactive dashboard using Microsoft Power BI for effective visualization and business decision-making.
- To generate meaningful insights that help improve sales performance, inventory management, and customer understanding.

3. TECHNOLOGIES USED

- **Microsoft Excel** - Used for data cleaning, formatting, sorting, filtering, and basic analysis using formulas, charts, and pivot tables.
- **SQL** - Used to query the retail sales dataset, perform aggregations, joins, and extract meaningful business insights from multiple tables.
- **Python** - Used for exploratory data analysis, data pre-processing, and visualization using libraries such as Pandas, NumPy, and Matplotlib.
- **Microsoft Power BI** - Used to create interactive dashboards, KPI reports, and visualizations for analyzing retail sales performance effectively.

4. TASKS

4.1 Excel – Tasks

1. Standardize Column Headers

- Column names across all sheets were standardized using a consistent naming format for better readability and analysis.
- This helped maintain uniformity while importing and processing the data in other tools.

2. Remove Duplicates

- Duplicate records were identified and removed from tables such as customers and order items.

3. Handle Missing Values

- Blank and null values were detected using filters.
- Missing fields like ZIP codes and phone numbers were replaced with placeholders such as “NA”.

4. Data Type Conversion

- Date columns such as order date and shipped date were converted into proper Excel date format.

5. Data Validation

- Dropdown lists were created for fields like order status to maintain consistency in data entry.
- Data -> Data validation -> list -> values(order placed, pending, shipped, delivered)

6. Create New Derived Columns

- A new column called total_price was created using formulas based on quantity, price, and discount.
- Formula: Total_price= (listprice*Quantity) - Discount

7. Merge Lookup Data

- VLOOKUP functions were used to combine product_name with order items table using product_id
Formula: =vlookup(lookup value, table array, column index, [range lookup]).

H2		=VLOOKUP([@[product_id]],[products.xlsx]products!\$A:\$B,2,FALSE)						
	A	B	C	D	E	F	G	H
1	order_id	item_id	product_id	quantity	list_price	discount	total_price	product_name
2	1	1	20	1.00	599.99	0.2	599.79	Electra Townie Original 7D EQ - Women's - 2016
3	1	2	8	2.00	1799.99	0.07	3599.91	Trek Remedy 29 Carbon Frameset - 2016
4	1	3	10	2.00	1549	0.05	3097.95	Surly Straggler - 2016
5	1	4	16	2.00	599.99	0.05	1199.93	Electra Townie Original 7D EQ - 2016
6	1	5	4	1.00	2899.99	0.2	2899.79	Trek Fuel EX 8 29 - 2016
7	2	1	20	1.00	599.99	0.07	599.92	Electra Townie Original 7D EQ - Women's - 2016

8. Create Basic Pivot Table

Pivot tables were created to summarize and analyze sales performance by product category.
Insert -> Pivot Table – Pivot Table Fields – Rows(product category id),value(Total Sale).

9. Sort and Filter for Outliers

- Sorting and filtering techniques were used to identify unusual values in pricing data.
- To find the outliers we need to find the,
Q1 (quartile 1) and Q3 (quartile 2)
IQR = Q3-Q1
Lower limit = Q1-1.5*IQR
Upper limit = Q3+1.5*IQR

G2		=IF(OR([@[list_price]]<\$157,[@[list_price]]>\$158),"outliers","Normal")								
	A	B	C	D	E	F	G	H	I	
1	product_id	product_name	brand_id	category_id	model_year	list_price	outliers			
2	1	Trek 820 - 2016	9	6	2016	379.99	Normal			
3	2	Ritchey Timberwol	5	6	2016	749.99	Normal			
4	3	Surly Wednesday F	8	6	2016	999.99	Normal	Q1	439.99	
5	4	Trek Fuel EX 8 29 -	9	6	2016	2899.99	Normal	Q3	2299.99	
6	5	Heller Shagamaw f	3	6	2016	1320.99	Normal	IQR	1860	
7	6	Surly Ice Cream Tri	8	6	2016	469.99	Normal	Lowerlimit	-2350.01	
8	7	Trek Slash 8 27.5 -	9	6	2016	3999.99	Normal	Upperlimit	5089.99	
9	8	Trek Remedy 29 Ci	9	6	2016	1799.99	Normal			
10	9	Trek Conduit+ - 20	9	5	2016	2999.99	Normal			
11	10	Surly Straggler - 20	8	4	2016	1549	Normal			

10. Prepare Final CSV Files

Cleaned datasets were exported and saved as CSV files for SQL database import.
File -> save as -> csv(comma delimited)

- brands.csv
- categories.csv
- customers.csv
- order_items.csv
- orders.csv
- products.csv
- staffs.csv
- stocks.csv
- stores.csv

4.2 SQL TASK

- Create a database retailsales.

```
create database retailsales;
use retailsales;
```

- To import the csv files into the database
Select the database -> Right Click on Table -> Table data import wizard -> choose the source of file
-> assign columns.
- To change the data type of the id's from int to varchar use alter command

```
-- change the data type (int to varchar)
alter table brands modify brand_id varchar(20);
alter table categories modify category_id varchar(20);
alter table customers modify customer_id varchar(30);
alter table order_items modify order_id varchar(30);
alter table order_items modify item_id varchar(30);
alter table order_items modify product_id varchar(30);
alter table orders modify order_id varchar(30);
alter table orders modify customer_id varchar(30);
alter table orders modify store_id varchar(30);
alter table orders modify staff_id varchar(30);
alter table products modify product_id varchar(30);
alter table products modify brand_id varchar(30);
alter table products modify category_id varchar(30);
alter table staffs modify staff_id varchar(30);
alter table staffs modify store_id varchar(30);
alter table staffs modify manager_id varchar(30);
alter table stocks modify store_id varchar(30);
alter table stocks modify product_id varchar(30);
alter table stores modify store_id varchar(30);
```

- Primary keys were added to the unique columns

```
-- add primary key
alter table brands add constraint new_constraint primary key (brand_id);
alter table categories add constraint new_constraint primary key (category_id);
alter table customers add constraint new_constraint primary key (customer_id);
alter table orders add constraint new_constraint primary key (order_id);
alter table products add constraint new_constraint primary key (product_id);
alter table staffs add constraint new_constraint primary key (staff_id);
alter table stores add constraint new_constraint primary key (store_id);
```

- Foreign keys were added to establish the relationships between tables.

```
-- add foreign key
alter table order_items add constraint new_constraint foreign key (order_id) references orders (order_id);
alter table order_items add constraint fk_constraint foreign key (product_id) references products (product_id);
alter table orders add constraint fk_cust_constraint foreign key (customer_id) references customers (customer_id);
alter table orders add constraint fk_store_constraint foreign key (store_id) references stores (store_id);
alter table orders add constraint fk_staff_constraint foreign key (staff_id) references staffs (staff_id);
alter table products add constraint fk_cate_constraint foreign key (category_id) references categories (category_id);
alter table products add constraint fk_brand_constraint foreign key (brand_id) references brands (brand_id);
alter table staffs add constraint fk_storeid_constraint foreign key (store_id) references stores (store_id);
alter table stocks add constraint fk_stor_constraint foreign key (store_id) references stores (store_id);
alter table stocks add constraint fk_product_constraint foreign key (product_id) references products (product_id);
```

- Joins & Group By

```
-- 3. Join orders, order_items, and products to display detailed line items.
select o.order_id, p.product_id, p.product_name, oi.quantity, oi.list_price, (oi.quantity*oi.list_price) as total_price from orders
as o inner join order_items as oi on o.order_id= oi.order_id inner join products as p on p.product_id=oi.product_id;

-- 4. Write a query to group sales (total_price) by each store_id.
select o.store_id,sum(oi.quantity*oi.list_price) as total_price from orders as o inner join order_items as oi on o.order_id=oi.order_id
group by store_id;

-- 5. Use ORDER BY and LIMIT to get the top 5 most sold products by quantity.
select order_id, product_id, product_name, quantity,list_price from order_items order by quantity desc limit 5;

-- 6. For each customer, return total orders placed, total items purchased, and total revenue.
select o.customer_id, count(oi.order_id) as total_orders_placed,sum(oi.quantity)as total_items, sum(oi.quantity * oi.list_price)
as total_revenue from order_items as oi inner join orders as o on oi.order_id=o.order_id group by customer_id;

-- 7. Write a query to classify customers into spending brackets (e.g., low, medium, high).
select o.customer_id, sum(oi.quantity*oi.list_price)as total_spending,
case
when sum(oi.quantity*oi.list_price)<5000 then 'Low'
when sum(oi.quantity*oi.list_price) between 5000 and 12000 then 'Medium'
else 'High'
end as category
from order_items as oi inner join orders as o on oi.order_id=o.order_id group by o.customer_id;

-- 8. Analyze total revenue generated by each staff member based on their handled orders.
select o.staff_id, sum(oi.quantity*oi.list_price) as total_revenue from orders as o
inner join order_items as oi on o.order_id=oi.order_id group by o.staff_id;

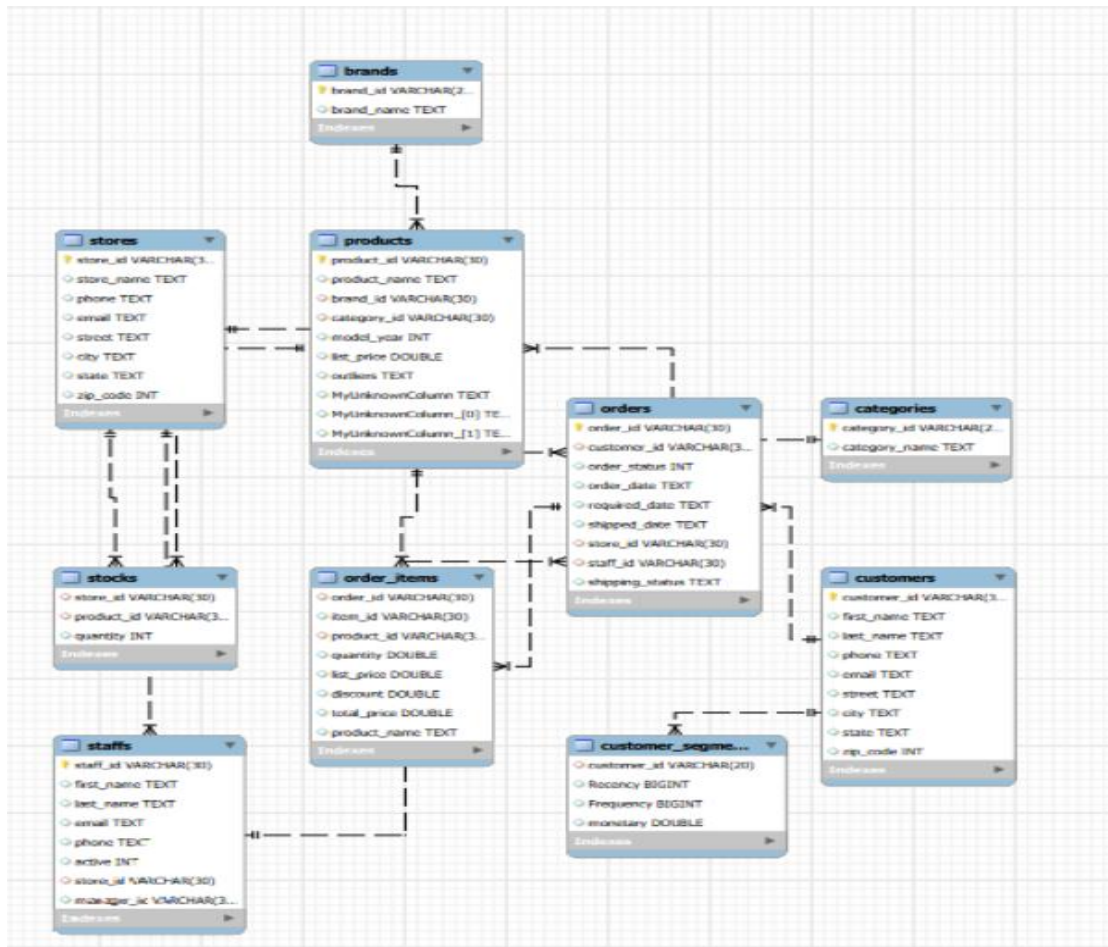
-- 9. Write a query to list products where stock quantity < 10 in any store.
select s.product_id, p.product_name, s.quantity from stocks as s inner join products as p on s.product_id=p.product_id where quantity<10;
```

Segmentation

```
-- 10. Create a table customer_segments that will be populated from Python ML results later.
alter table customer_segments modify customer_id varchar(20);
alter table customer_segments add constraint fk_cus_constraint foreign key(customer_id) references customers(customer_id);
select * from customer_segments;
```

ER DIAGRAM

Table	Primary Key	Foreign Key
Brand	Brand_id	
Category	Category_id	
Customers	Customer_id	
Orders	Order_id	Customer_id, store_id, staff_id
Order_items		Order_id, product_id
Products	Product_id	Category_id, brand_id
Stocks		Store_id, product_id
Stores	Store_id	
Staffs	Staff_id	Store_id



4.3 PYTHON TASK

Import data from SQL

```
pip install pandas
```

```
import pandas as pd
```

```
pip install sqlalchemy
```

```
from sqlalchemy import create_engine
engine = create_engine("mysql+pymysql://root:1234@localhost:3306/retailsales")
```

```
orders = pd.read_sql("SELECT * FROM orders", engine)
```

```
orders.head(10)
```

	order_id	customer_id	order_status	order_date	required_date	shipped_date	store_id	staff_id	shipping_status
0	1	259	4	01-01-2016	03-01-2016	03-01-2016	1	2	Delivered
1	10	442	4	05-01-2016	06-01-2016	06-01-2016	2	6	Delivered
2	100	1237	4	01-03-2016	04-03-2016	03-03-2016	2	7	Delivered
3	1000	645	4	12-07-2017	14-07-2017	15-07-2017	2	7	Delivered
4	1001	354	4	13-07-2017	16-07-2017	15-07-2017	2	6	Delivered
5	1002	1418	4	14-07-2017	16-07-2017	16-07-2017	1	3	Delivered
6	1003	503	4	14-07-2017	16-07-2017	16-07-2017	2	7	Delivered
7	1004	1389	4	14-07-2017	17-07-2017	16-07-2017	2	6	Delivered
8	1005	266	4	15-07-2017	17-07-2017	16-07-2017	2	7	Delivered
9	1006	1070	4	15-07-2017	18-07-2017	17-07-2017	2	7	Delivered

```
order_items = pd.read_sql("SELECT * FROM order_items",engine)
```

```
order_items.head(10)
```

	order_id	item_id	product_id	quantity	list_price	discount	total_price	product_name
0	1	1	20	1.0	599.99	0.20	599.79	Electra Townie Original 7D EQ - Women's - 2016
1	1	2	8	2.0	1799.99	0.07	3599.91	Trek Remedy 29 Carbon Frameset - 2016
2	1	3	10	2.0	1549.00	0.05	3097.95	Surly Straggler - 2016
3	1	4	16	2.0	599.99	0.05	1199.93	Electra Townie Original 7D EQ - 2016
4	1	5	4	1.0	2899.99	0.20	2899.79	Trek Fuel EX 8 29 - 2016
5	2	1	20	1.0	599.99	0.07	599.92	Electra Townie Original 7D EQ - Women's - 2016
6	2	2	16	2.0	599.99	0.05	1199.93	Electra Townie Original 7D EQ - 2016
7	3	1	3	1.0	999.99	0.05	999.94	Surly Wednesday Frameset - 2016
8	3	2	20	1.0	599.99	0.05	599.94	Electra Townie Original 7D EQ - Women's - 2016
9	4	1	2	2.0	749.99	0.10	1499.88	Ritchey Timberwolf Frameset - 2016

```
customers=pd.read_sql("SELECT * FROM customers",engine)
```

```
customers.head(10)
```

	customer_id	first_name	last_name	phone	email	street	city	state	zip_code
0	1	Debra	Burks	NA	debra.burks@yahoo.com	9273 Thorne Ave.	Orchard Park	NY	14127
1	10	Pamelia	Newman	NA	pamelia.newman@gmail.com	476 Chestnut Ave.	Monroe	NY	10950
2	100	Kellie	Franco	NA	kellie.franco@yahoo.com	444 South Walnut Rd.	Commack	NY	11725
3	1000	Shiloh	Reeves	NA	shiloh.reeves@msn.com	818 Kirkland Lane	West Babylon	NY	11704
4	1001	Lee	Roman	(631) 913-6967	lee.roman@gmail.com	4 Canal Ave.	Brentwood	NY	11717
5	1002	Cami	Williamson	NA	camli.williamson@hotmail.com	9511A Windsor Drive	Massapequa	NY	11758
6	1003	Sharell	Ross	NA	sharell.ross@msn.com	7830 Devonshire Ave.	Vista	CA	92083
7	1004	Jerald	Blackwell	NA	gerald.blackwell@hotmail.com	8364 South Wakehurst Ave.	East Elmhurst	NY	11369
8	1005	Ruthanne	Franco	NA	ruthanne.franco@yahoo.com	61 Applegate St.	Astoria	NY	11102
9	1006	Marjory	Leonard	NA	marjory.leonard@msn.com	800 Greystone Lane	Apple Valley	CA	92307

Data Summarizing

- Describe() - Used to get statistical summary of numerical columns in a dataset.

```
orders.describe()
```

order_status	
count	1615.000000
mean	3.778947
std	0.693430
min	1.000000
25%	4.000000
50%	4.000000
75%	4.000000
max	4.000000

```
order_items.describe()
```

	quantity	list_price	discount	total_price
count	4722.000000	4722.000000	4722.000000	4722.000000
mean	1.498941	1212.707872	0.105373	1816.707181
std	0.500052	1352.798257	0.058119	2228.162327
min	1.000000	89.990000	0.050000	89.790000
25%	1.000000	429.000000	0.050000	539.780000
50%	1.000000	599.990000	0.085000	939.910000
75%	2.000000	1549.000000	0.200000	1999.910000
max	2.000000	11999.990000	0.200000	23999.880000

```
customers.describe()
```

	zip_code
count	1445.000000
mean	34200.015225
std	34733.933551
min	10002.000000
25%	11419.000000
50%	11967.000000
75%	76110.000000
max	95993.000000

Info() - Used to get basic information about the dataset.

```
orders.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1615 entries, 0 to 1614
Data columns (total 9 columns):
#   Column             Non-Null Count  Dtype
---  -
0   order_id           1615 non-null   object
1   customer_id        1615 non-null   object
2   order_status       1615 non-null   int64
3   order_date         1615 non-null   object
4   required_date      1615 non-null   object
5   shipped_date        1445 non-null   object
6   store_id           1615 non-null   object
7   staff_id           1615 non-null   object
8   shipping_status    1615 non-null   object
dtypes: int64(1), object(8)
memory usage: 113.7+ KB
```

```
order_items.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4722 entries, 0 to 4721
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0   order_id        4722 non-null   object
1   item_id         4722 non-null   object
2   product_id      4722 non-null   object
3   quantity        4722 non-null   float64
4   list_price      4722 non-null   float64
5   discount        4722 non-null   float64
6   total_price     4722 non-null   float64
7   product_name    4722 non-null   object
dtypes: float64(4), object(4)
memory usage: 295.3+ KB
```

```
customers.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1445 entries, 0 to 1444
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   customer_id     1445 non-null   object
1   first_name      1445 non-null   object
2   last_name       1445 non-null   object
3   phone          1445 non-null   object
4   email          1445 non-null   object
5   street          1445 non-null   object
6   city           1445 non-null   object
7   state          1445 non-null   object
8   zip_code       1445 non-null   int64
dtypes: int64(1), object(8)
memory usage: 101.7+ KB
```

Value counts() - Used to count unique values in a column.

```
orders.value_counts()
```

order_id	customer_id	order_status	order_date	required_date	shipped_date	store_id	staff_id	shipping_status	
1	259	4	01-01-2016	03-01-2016	03-01-2016	1	2	Delivered	1
561	90	4	19-11-2016	22-11-2016	22-11-2016	2	6	Delivered	1
56	1241	4	04-02-2016	06-02-2016	07-02-2016	2	6	Delivered	1
559	959	4	16-11-2016	17-11-2016	17-11-2016	1	2	Delivered	1
558	1443	4	15-11-2016	16-11-2016	17-11-2016	2	7	Delivered	1
144	1124	4	26-03-2016	29-03-2016	29-03-2016	2	7	Delivered	1
1439	463	4	15-03-2018	16-03-2018	17-03-2018	2	7	Delivered	1
1438	263	4	15-03-2018	17-03-2018	16-03-2018	2	6	Delivered	1
1437	518	4	15-03-2018	16-03-2018	16-03-2018	1	2	Delivered	1
999	590	4	12-07-2017	13-07-2017	14-07-2017	2	6	Delivered	1

Name: count, Length: 1445, dtype: int64

```
order_items.value_counts()
```

order_id	item_id	product_id	quantity	list_price	discount	total_price	product_name	
1	1	20	1.0	599.99	0.20	599.79	Electra Townie Original 7D EQ - Women's - 2016	1
519	5	7	1.0	3999.99	0.05	3999.94	Trek Slash 8 27.5 - 2016	1
524	3	9	1.0	2999.99	0.20	2999.79	Trek Conduit+ - 2016	1
	2	2	1.0	749.99	0.05	749.94	Ritchey Timberwolf Frameset - 2016	1
	1	11	1.0	1680.99	0.07	1680.92	Surly Straggler 650b - 2016	1
1478	2	123	1.0	999.99	0.20	999.79	Trek Farley Carbon Frameset - 2018	1
	1	100	2.0	489.99	0.10	979.88	Electra Townie 3i EQ (20-inch) - Boys' - 2017	1
1477	1	210	2.0	1549.00	0.20	3097.80	Surly Straggler - 2018	1
1476	5	265	2.0	289.99	0.10	579.88	Strider Strider 20 Sport - 2018	1
999	2	106	1.0	449.99	0.05	449.94	Sun Bicycles Cruz 3 - 2017	1

Name: count, Length: 4722, dtype: int64

```
customers.value_counts()
```

customer_id	first_name	last_name	phone	email	street	city	state	zip_code	
1	Debra	Burks	NA	debra.burks@yahoo.com	9273 Thorne Ave.	Orchard Park	NY	14127	1
572	Wm	Pope	NA	wm.pope@msn.com	212 Marsh Court	Auburn	NY	13021	1
570	Marissa	Summers	NA	marissa.summers@msn.com	81 Carpenter St.	Saint Albans	NY	11412	1
57	Pamala	Henry	NA	pamala.henry@aol.com	197 Bridgeton Ave.	Bronx	NY	10451	1
569	Lucas	Estes	NA	lucas.estes@yahoo.com	99 Old Church St.	Santa Monica	CA	90403	1
1430	Tayna	Wade	(661) 858-1224	tayna.wade@hotmail.com	65 Smith Store Dr.	Bakersfield	CA	93306	1
143	Janae	Doyle	NA	janae.doyle@gmail.com	760 Rose Ave.	Jackson Heights	NY	11372	1
1429	Lorrie	Justice	NA	lorrie.justice@gmail.com	8396 Brook Drive	Pomona	CA	91768	1
1428	Carola	Johns	NA	carola.johns@msn.com	7077 Fordham Rd.	Santa Cruz	CA	95060	1
999	Latoya	Johns	NA	latoya.johns@hotmail.com	7914 W. Woodsman St.	Ballston Spa	NY	12020	1

Name: count, Length: 1445, dtype: int64

Matplotlib / Seaborn

Matplotlib - Python library used for creating graphs, charts, and data visualizations.

Install the library matplotlib and import it

```
pip install matplotlib
```

```
import matplotlib.pyplot as plt
```

Product wise price

```
productwiseprice=order_items.groupby('item_id')['total_price'].sum()
```

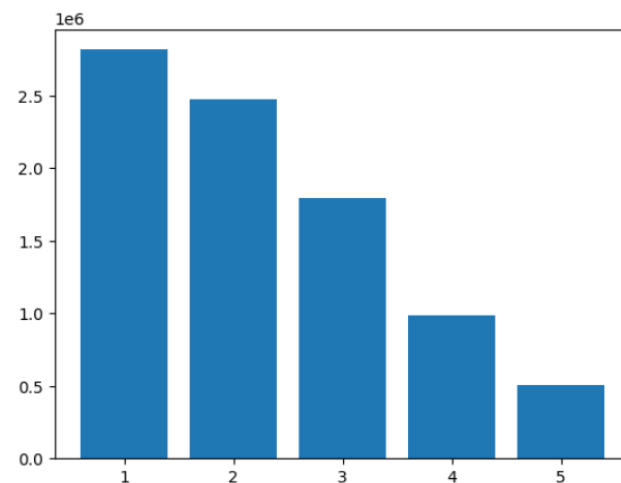
```
productwiseprice=productwiseprice.reset_index()
```

```
productwiseprice
```

	item_id	total_price
0	1	2815457.65
1	2	2472966.62
2	3	1796232.32
3	4	983999.69
4	5	509835.03

```
plt.bar(productwiseprice['item_id'],productwiseprice['total_price'])
```

<BarContainer object of 5 artists>



Item wise list price

```
order_sts=order_items.groupby('item_id')['list_price'].sum()
```

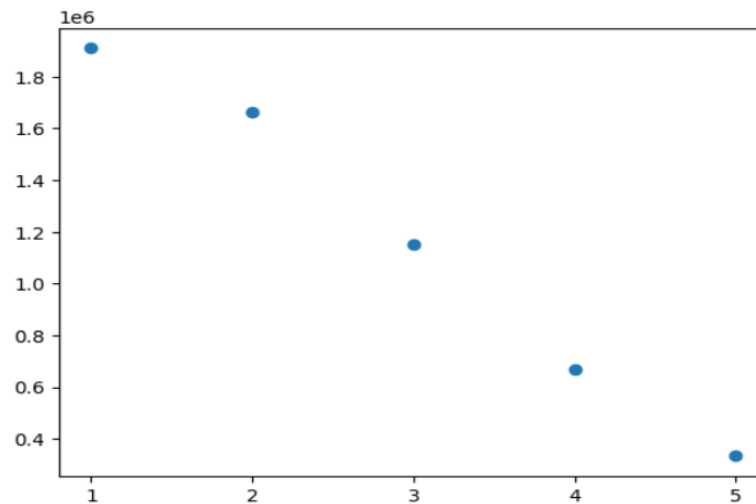
```
order_sts=order_sts.reset_index()
```

```
order_sts
```

	item_id	list_price
0	1	1909769.98
1	2	1661078.81
2	3	1153049.35
3	4	669959.68
4	5	332548.75

```
plt.scatter(order_sts['item_id'],order_sts['list_price'])
```

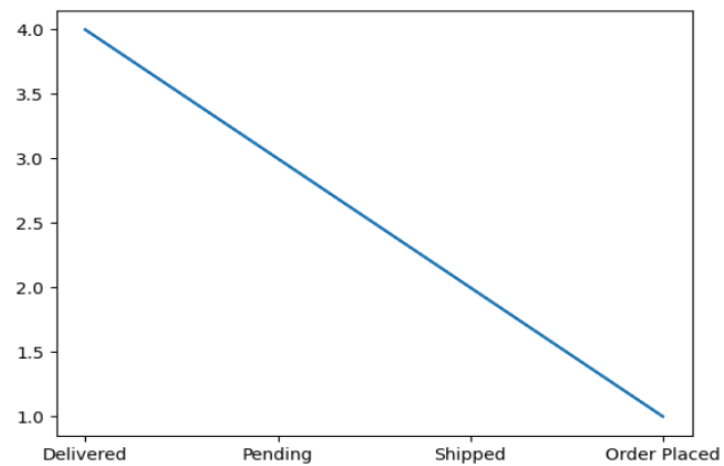
```
<matplotlib.collections.PathCollection at 0x26373d77010>
```



Order Status wise Shipping status

```
plt.plot(orders['shipping_status'],orders['order_status'])
```

```
<matplotlib.lines.Line2D at 0x26373da5810>
```



Seaborn – Python library used for creating attractive and informative statistical graphs. It is built on top of Matplotlib.

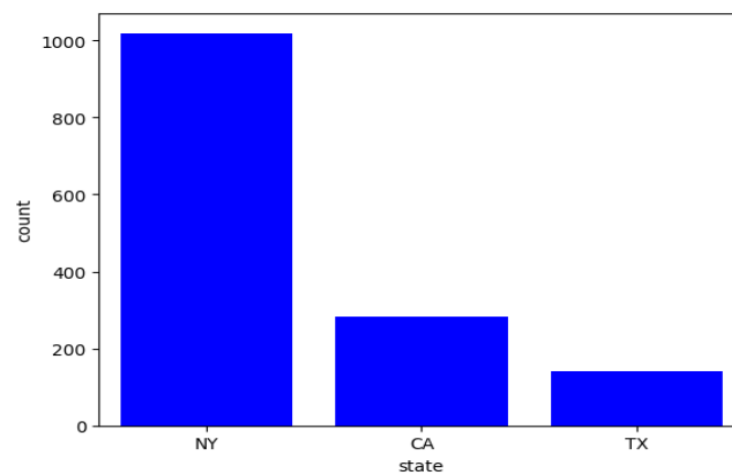
```
pip install seaborn
```

```
import seaborn as sns
```

State wise Customers

```
sns.countplot(data=customers, x='state', facecolor='blue')
```

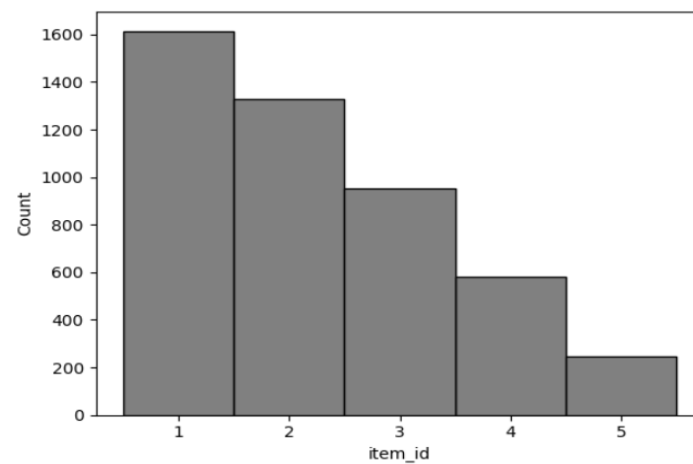
<Axes: xlabel='state', ylabel='count'>



Item id wise order items

```
sns.histplot(data=order_items, x='item_id', facecolor='grey')
```

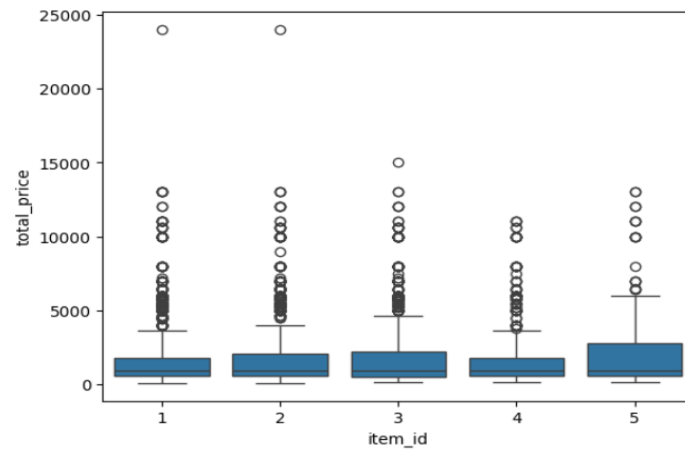
<Axes: xlabel='item_id', ylabel='Count'>



Item_id wise total price

```
sns.boxplot(data=order_items, x='item_id', y='total_price')
```

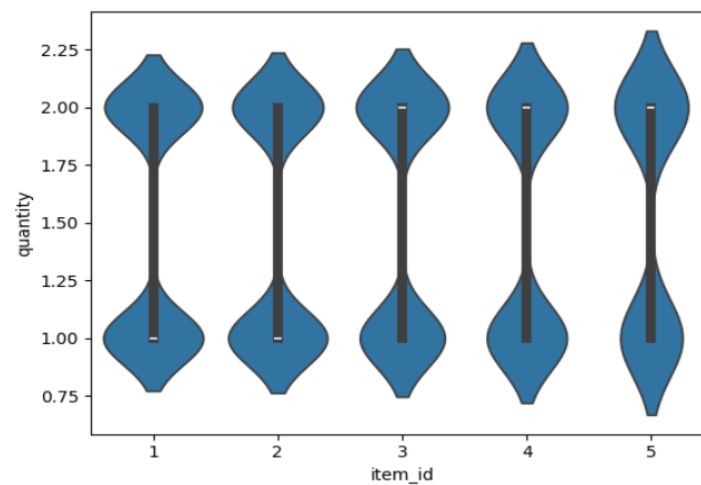
<Axes: xlabel='item_id', ylabel='total_price'>



Item id wise quantity

```
sns.violinplot(data=order_items, x='item_id', y='quantity')
```

<Axes: xlabel='item_id', ylabel='quantity'>



Date conversion

```
df=orders[['customer_id','order_date']]
```

df

	customer_id	order_date
0	259	01-01-2016
1	442	05-01-2016
2	1237	01-03-2016

```
df.loc[:, 'order_date'] = pd.to_datetime(df['order_date'], format='%d-%m-%Y')
```

```
df['order_date'] = pd.to_datetime(df['order_date'], format='%d-%m-%Y', errors='coerce')
```

RFM

Recency: Days since last order

Frequency: Number of orders

Monetary: Total value of all purchases

Recency

```
df['Recency']=(current_date-df['order_date']).dt.days  
print(df)
```

	customer_id	order_date	Recency
0	259	2016-01-01	3766
1	442	2016-01-05	3762
2	1237	2016-03-01	3706
3	645	2017-07-12	3208
4	354	2017-07-13	3207
...	...	...	...
1610	621	2017-07-11	3209
1611	915	2017-07-11	3209
1612	485	2017-07-11	3209
1613	219	2017-07-12	3208
1614	590	2017-07-12	3208

Frequency

```
orderwisecount=orders.groupby('customer_id')['order_id'].agg(Frequency='count')
```

orderwisecount

Frequency	
customer_id	
1	3
10	3
100	1
1000	1
1001	1

Merge orders and order_items

```
merge_data=pd.merge(orders,order_items,on='order_id',how='inner')
```

merge_data

	order_id	customer_id	order_status	order_date	required_date	shipped_date	store_id	staff_id	shipping_status	item_id	product_id	quantity	list_price	discount
0	1	259	4	01-01-2016	03-01-2016	03-01-2016	1	2	Delivered	1	20	1.0	599.99	0.
1	1	259	4	01-01-2016	03-01-2016	03-01-2016	1	2	Delivered	2	8	2.0	1799.99	0.

Monetary

```
merge_data['monetary']=merge_data['quantity']*merge_data['list_price']
```

Merge the recency, frequency, monetary to the rfm table

```
rfm=pd.merge(pd.merge(df,orderwisecount,on='customer_id'),orderwisepurchase,on='customer_id')
```

rfm

	customer_id	order_date	Recency	Frequency	monetary
0	259	2016-01-01	3766	1	11397.94
1	442	2016-01-05	3762	1	269.99
2	1237	2016-03-01	3706	1	14761.90
3	645	2017-07-12	3208	1	4062.95
4	354	2017-07-13	3207	1	7830.95
...	...	...	...	...	...
1610	621	2017-07-11	3209	1	1749.96
1611	915	2017-07-11	3209	1	1369.97
1612	485	2017-07-11	3209	1	1907.94

Remove the order_date column

```
rfm.drop('order_date',axis=1,inplace=True)
```

rfm

	customer_id	Recency	Frequency	monetary
0	259	3766	1	11397.94
1	442	3762	1	269.99
2	1237	3706	1	14761.90
3	645	3208	1	4062.95
4	354	3207	1	7830.95

Export the result to SQL

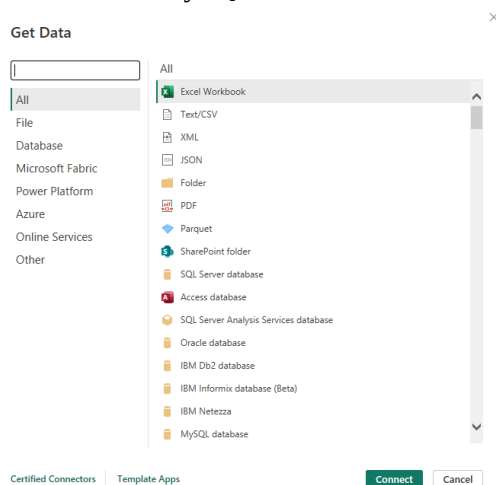
```
rfm.to_sql('customer_segments', con=engine, if_exists='replace', index=False)
```

```
rfm.to_sql('customer_segments', con=engine, if_exists='replace', index=False)
```

1615

4.4 POWER BI TASK

- Install MySQL Connector ODBC to connect MySQL.
- Import data from SQL
 - Get data -> MySQL Database -> connect



- Enter the server name and database name click ok.

MySQL database

Server
127.0.0.1

Database
retailsales

> Advanced options

OK Cancel

- Database connected and the tables within the database are listed select the tables and click transform data.

Navigator

Display Options +

127.0.0.1: retailsales [10]

- retailsales.brands
- retailsales.categories
- retailsales.customer_segments
- retailsales.customers
- retailsales.order_items
- retailsales.orders
- retailsales.products
- retailsales.staffs
- retailsales.stocks
- retailsales.stores

retailsales.brands
Preview downloaded on 06 May 2026

brand_id	brand_name	retailsales.products
1	Electra	Table
2	Haro	Table
3	Heller	Table
4	Pure Cycles	Table
5	Ritchey	Table
6	Strider	Table
7	Sun Bicycles	Table
8	Surly	Table
9	Trek	Table

Select Related Tables

Load Transform Data Cancel

- Check the accuracy of the data and click close and apply.

Retailsalesproject

Home Transform Add Column View Tools Help

Close & Apply ~ New Source ~ Recent Sources ~ Enter Data ~ Data source settings ~ Manage Parameters ~ Export query results ~ Refresh Preview ~ Properties ~ Advanced Editor ~ Manage ~ Chk Colu

Queries [10]

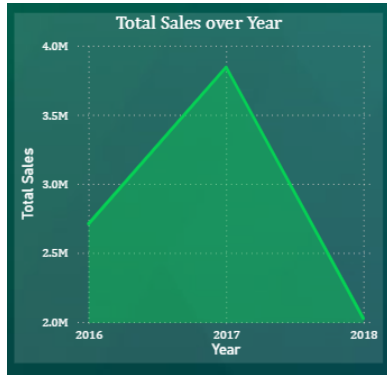
This preview may be up to 24 days old. Refresh

retailsales.brands

Table.TransformColumnTypes(retailsales.brands,{"brand_id", Int

brand_id	brand_name	retailsales.products
1	Electra	Table
2	Haro	Table
3	Heller	Table
4	Pure Cycles	Table
5	Ritchey	Table
6	Strider	Table
7	Sun Bicycles	Table
8	Surly	Table
9	Trek	Table

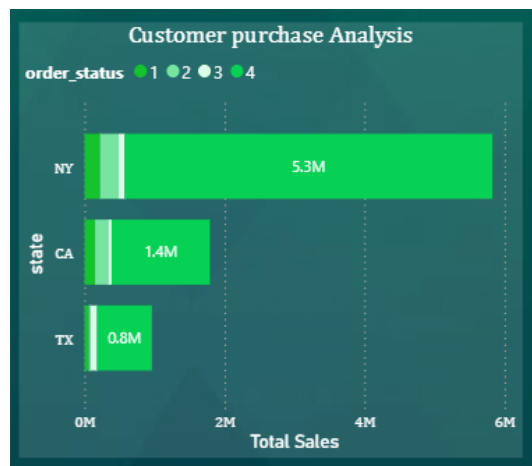
Sales Overview



Top Selling Products



Customer purchase analysis



Sales by store



Staff performance

first_name	Total Sales
Genna	9,52,722.26
Kali	5,16,695.17
Layla	4,45,905.59
Marcelene	29,38,888.73
Mireya	8,37,423.65
Venita	28,87,353.48
Total	85,78,988.88

DASHBOARD

Charts

- Clustered column chart
 - Stacked bar chart
 - Area chart
 - Donut chart
 - Map
 - Stacker column chart
 - Clustered bar chart
 - Table
 - Waterfall chart
 - Slicers
 - Cards
- Cards- Cards are used to display important business metrics in a simple and clear format.
 - Slicers - Slicers are interactive filtering tools that allow users to view specific data based on selected criteria.
 - Buttons- Buttons are used to perform actions such as navigating between pages, resetting filters.

- Page Navigation - Page navigation is used to move seamlessly between different dashboard pages.

Home Dashboard

- Provides an overview of the Retail Sales Analysis project, highlighting key objectives, features, and the different analytical sections available in the dashboard.



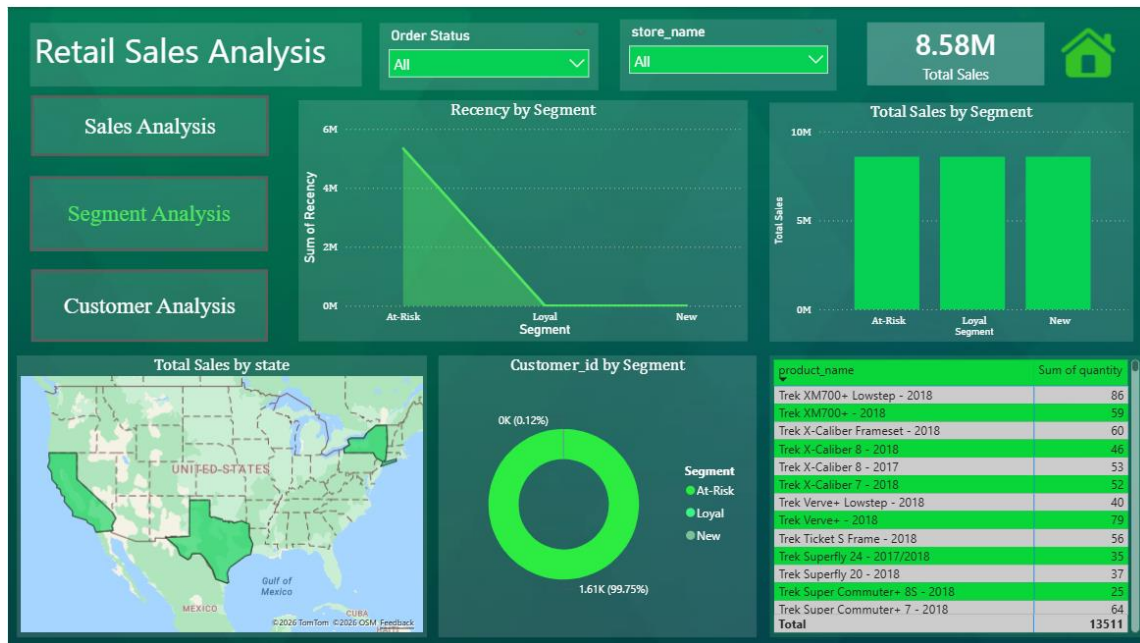
Sales Analysis Dashboard

- Displays sales performance metrics, trends, and product-wise sales insights to help monitor revenue growth and business performance.



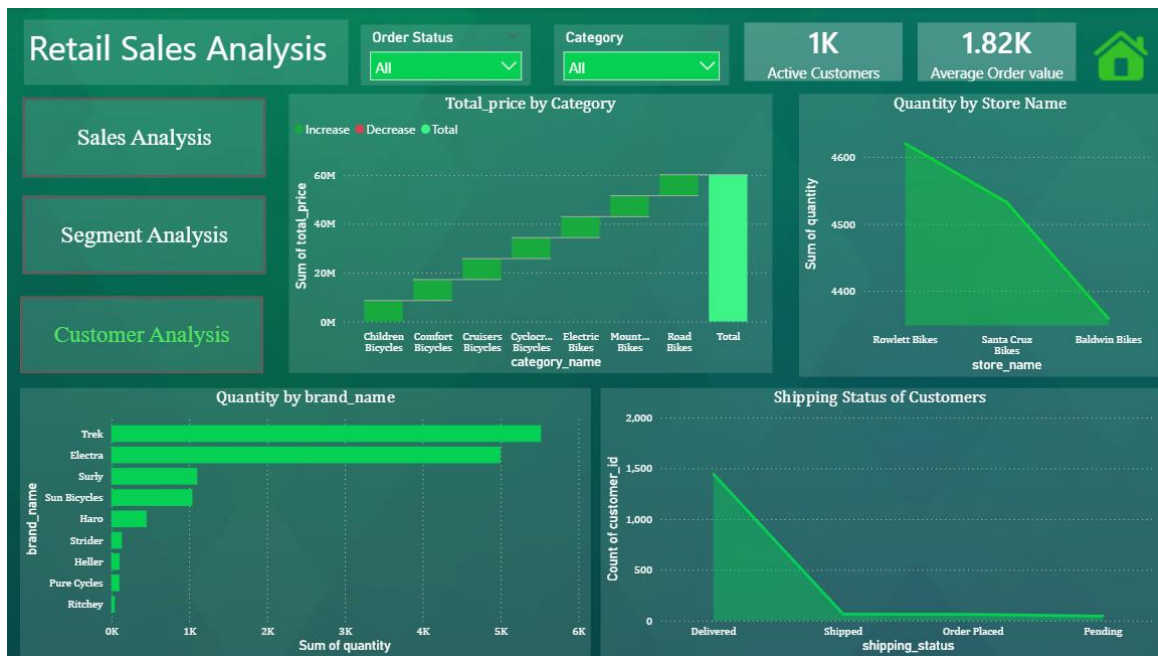
Segment Analysis Dashboard

- Analyzes customer segments, regional sales distribution, and segment-wise performance to identify valuable customer groups and market opportunities.



Customer Analysis Dashboard

- Presents customer behavior, purchasing patterns, order values, and shipping status insights to support customer-focused business decisions.



5. KEY INSIGHTS

- The business generated approximately **8.58M in total sales**, indicating strong overall revenue performance.
- Sales peaked during certain months, particularly around **April**, showing the impact of seasonal demand on revenue.
- A few products contributed significantly to total sales, helping identify the most profitable product categories.
- The **Loyal customer segment** represented the largest share of customers, highlighting the importance of customer retention strategies.
- States such as **New York (NY)** generated higher sales compared to other regions, indicating strong market demand.
- Brands like **Trek** contributed the highest sales quantity, making them key revenue drivers for the business.
- The dashboard shows around **1K active customers**, providing insight into customer engagement and purchasing behavior.
- Most orders were successfully delivered, reflecting an efficient order fulfillment process.
- Certain stores outperformed others in terms of sales and quantity sold, helping identify high-performing locations.
- The dashboard enables quick analysis of sales, customer behavior, and product performance, supporting informed business decisions and strategic planning.

6. CONCLUSION

This Retail Sales Analysis project provides valuable insights into sales trends, customer behavior, and product performance. The analysis shows that total sales reached **8.58M**, with **April recording the highest sales**, indicating a strong seasonal impact on business performance. Products from the **Trek brand** emerged as major revenue contributors, while loyal customers formed the largest customer segment. The dashboard enables businesses to identify high-performing products, regions, and customer groups, helping improve decision-making and operational efficiency. Overall, this project demonstrates how data-driven analysis can support strategic planning, enhance customer satisfaction, and drive sustainable business growth.